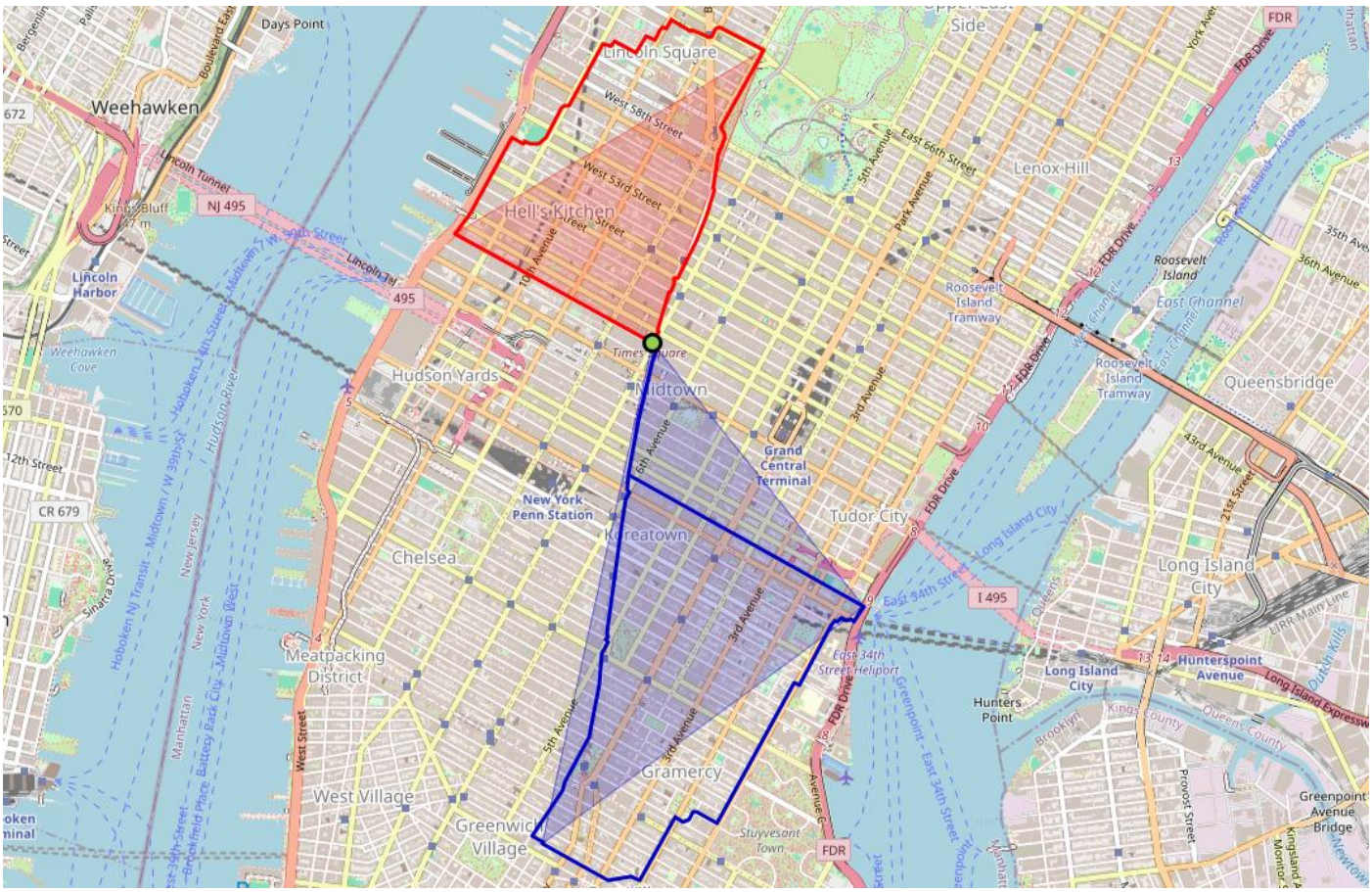
Algorithm for the k-length Round Trip problem.

R. Lewis, School of Mathematics, Cardiff University, Wales,
www.rhydlewis.eu, LewisR9@cardiff.ac.uk

P. Corcoran, School of Computer Science and Informatics, Cardiff University, Wales,
corcoranp@cardiff.ac.uk, https://www.cardiff.ac.uk/people/view/155896-corcoran-padraig

Last Updated: Thursday, 22 May 2025



1. Summary

This document contains instructions on how to use the algorithm implementation used in the following publication:

Lewis, R. and P. Corcoran (2024) "Fast Algorithms for Computing Fixed-Length Round Trips in in Real-World Street Networks". Springer Nature Computer Science, vol. 5, 868 https://link.springer.com/article/10.1007/s42979-024-03223-3

These implementations can be downloaded from https://zenodo.org/doi/10.5281/zenodo.8154412. The program is written in Python 3.8 and, for the above paper, was executed using an Anaconda 3, 64-bit installation. Full descriptions of these algorithms can be found in the above publication. An Anaconda distribution can be downloaded for free at https://www.anaconda.com/products/distribution

2. Algorithm Usage

The **main()** procedure for this implementation is found in the file **makeroundtrip.py**. The program is therefore executed from the command line using the following command:

```
> python makeroundtrip.py
```

This implementation allows the specification of several user-defined parameters. In the above command, default values have been used for all of these. Full usage instructions are viewed using the command:

```
> python makeroundtrip --help
```

This results in the following output.

```
usage: makeroundtrip.py [-h] [-s S S] [-k K] [-a A A] [-t {1,2,3}] [-n N] [-o {0,1,2,3}] [-seed SEED] [-name NAME] [-
r] [-v]

Fixed-length route generator using OpenRouteService API (R. Lewis, Cardiff University, 2023)

Optional arguments:
  -h, --help    show this help message and exit
  -s S S        start lon-lat coordinates
                        Default = -3.719126 51.486733
  -k K          desired length in meters
                        Default = 5000
  -a A A        algorithm choice and associated parameter x.
                        1 x = Isochrone-polygon (IP) method with multiple routes per call (for x calls);
                        2 x = IP method. Generate x polygons and get each route via a separate call;
                        3 x = OpenRouteService round_trip function (execute for x seeds/calls)
                        4 x = Do IP method and then do x-1 further rounds of linear regression learning
                        Default = 1 1
  -t {1,2,3}    travel mode:
                        1 = driving-car
                        2 = cycling-regular
                        3 = foot-walking
                        Default = 1
  -n N          number of points on each polygon (min = 3)
                        Default = 3
  -o {0,1,2,3}  optimisation option for stage 2
                        0 = no optimisation
                        1 = local search using BFS
                        2 = local search using RFS
                        3 = Johnson's algorithm
                        Default = 0
  -seed SEED    random seed used with local search procedure
                        Default = 1
  -name NAME    name of the output .html files.
                        Default = out
  -r            remove out-and-backs
  -v            show NetworkX visualisations of smoothed graph (requires user interaction)
```

As an example, consider the following command:

```
> python makeroundtrip.py -s -3.178475 51.489228 -k 10000 -t 2 -n 4 -o 1 -name CardiffMaths -r
```

This seeks 10km round trip solutions starting at the GPS (lon/lat) coordinate [-3.178475, 51.489228] (the location of Cardiff University's mathematics department in Wales). Other options used here include using cycling as the mode of transport (**-t 2**), four coordinates per polygon (**-n 4**), local search option 1 (**-o 1**), removing back-and-forths (**-r**), and outputting the solution visualisations to the file **CardiffMaths.html**. The full meaning of these terms can be found by consulting the above publication.

During execution, self-explanatory output is also written to the screen. Here is the output from the above command:

```
Input parameters   = {'s': [-3.178475, 51.489228], 'k': 10000, 'a': [1, 1], 't': 2, 'n': 4, 'o': 1, 'seed': 1, 'name':
'CardiffMaths', 'r': True, 'v': False}

Value for k'       = 10000
Initial Bearing    = 72.33611974160719
Num. polygons made = 12

Calculating routes 0 to 11 using isochrone method
Solutions written to html file: CardiffMaths
Cost of these solutions =
[[5215, 0], [6512, 0], [7594, 3], [6459, 2], [5555, 0], [5492, 2], [5410, 0], [5112, 11], [7202, 2], [8901, 0], [6203,
0], [8164, 3]]

Smoothed graph has n = 115 vertices and m = 216 arcs (containing 45 double ended edges)
```

```
Starting archive set has costs =
[[5215, 0], [5112, 11]]

Running Local Search algorithm, Option 1...
...Done.

Final archive set has costs =
[[26, 46], [35, 45], [57, 44], [76, 6], [77, 3], [185, 2], [1006, 1], [1191, 0]]

Solutions written to html file: CardiffMathsFront

Total runtime          = 1.6049940586090088 s.
Runtime without calls  = 0.39408111572265625 s.
Number of calls        = 2
```

In addition to textual output, interactive visualisations are also created in **html** files after each run. These can be viewed through a regular web browser.

3. API Keys

To help users get started with these algorithms, the authors have included their personal OpenRouteService API key in the source code. We ask that you replace this with your own key as soon as possible by registering for a free account with OpenRouteService (https://openrouteservice.org/). Once you have registered, you can generate a key and replace the value at the top of the **config.py** file.

4. Copyright Notice

Redistribution and use of this source code, with or without modification, are permitted provided that a citation is made to the publication given at the start of this document. Neither the name of the University nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission. This software is provided by the contributors "as is"" and any express or implied warranties, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose are disclaimed. In no event shall the contributors be liable for any direct, indirect, incidental, special, exemplary, or consequential damages (including, but not limited to, procurement of substitute goods or services; loss of use, data, or profits; or business interruption) however caused and on any theory of liability, whether in contract, strict liability, or tort (including negligence or otherwise) arising in any way out of the use of this software, even if advised of the possibility of such damage. This software is supplied without any support services.